

Chapter 1: AI-Ready Data Starts Locally

Preparation For AI: From Raw Data To Reliable Models

Alexander D'hoore

AI-Ready Data Starts Locally

AI starts before the model.

The first professional skill is not choosing the most advanced algorithm.

It is learning to ask whether the data path is understandable enough to trust a model result.

raw data -> assumptions -> checks -> split -> model -> evidence

Before we build a data platform, we learn to distrust two things:

- mysterious data
- too-good metrics

Small local files make both visible.

Four Habits

The labs are about four habits:

- inspect the data before modelling
- make expectations explicit
- validate before training
- judge the split before trusting the metric

Everything else in the chapter supports those habits.



What Makes Data AI Ready?

AI-ready data is not just data a model can read.

It is data that another person can:

- understand
- validate
- rebuild
- connect to a model result



Data Does Not Have One Shape

AI projects may start from:

- rows and columns
- sensor streams
- text documents
- multimedia assets
- scientific or spatial files

The data shape changes the preparation strategy.



Shape Changes The Questions

A table asks:

- what does each column mean?
- what is the target?
- which rows belong together?

An asset dataset asks:

- where are the files?
- where are the labels?
- how do we know nothing changed?



Expectations Become Checks

Before preparing data, define what should arrive:

- files
- columns or fields
- data types
- allowed values
- labels
- identifiers

The same expectation can later become code.

```
required = {"elevation", "slope",  
            "cover_type"}  
missing = required - set(forest.columns)
```

Fail Fast

A failed check is not automatically bad.

Raw data can violate expectations.

The professional move is to make the violation visible before model training.

A fail-fast system:

- stops early
- shows the reason
- makes debugging smaller

It is better to stop than to quietly do the wrong thing.

Train, Validation, And Test

Good model evidence separates three roles:

- train: data the model learns from
- validation: data used while decisions are still being made
- test: data kept aside for the final estimate

Always ask:

What exactly was held out?

If the held-out data is not meaningful, the metric is not meaningful.

Random Splits Are Not Always Honest

Random rows can leak information in:

- time series
- machine histories
- multimedia datasets

Example: a gesture-recognition dataset with one person.

If the same person appears in train and test, the model may recognize that person instead of learning gestures that generalize to new people.



Reproducibility Needs The Chain

Another person should be able to trace the data, code, and transforms that led to a result.

AI-ready work connects:

- raw source data
- cleaned records
- labels and targets
- metadata
- feature definitions
- validation results
- model metrics and artifacts

Keep Raw Data Around

Raw data is evidence, not temporary input.

Teams are tempted to delete raw data after cleaning.

That hurts reproducibility.

Keep raw data around, even on slower storage, for:

- debugging
- understanding
- rebuilding cleaned data

Cleaned records make the data usable. Labels deserve special care.

Metadata And Features

Metadata removes ambiguity:

- source system
- export time
- units
- sensor type
- license
- privacy class

Feature definitions explain how model inputs were calculated.

Validation Before Training

Validation checks catch broken data before it becomes a broken model.

Examples:

- row counts
- missing values
- schema checks
- allowed labels
- checksums

Later, pipelines can automate this. That is one of the best uses of Airflow.

Three Reliability Questions

When a team says a dataset is ready for AI, ask:

1. Can we explain where it came from?
2. Can we rebuild it?
3. Can we notice when it changes?

Later in the course:

- rebuild it: pipelines
- notice changes: manifests and hashes
- explain origins: metadata and layers

Lab 1: Forest Cover Type

Each row describes one forest land patch.

The task:

cartographic measurements -> forest cover type

The dataset has seven classes, including Spruce/Fir, Lodgepole Pine, Ponderosa Pine, Cottonwood/Willow, Aspen, Douglas-fir, and Krummholz.

The question is whether the accuracy is believable.



Lab 2: Turbofan Degradation

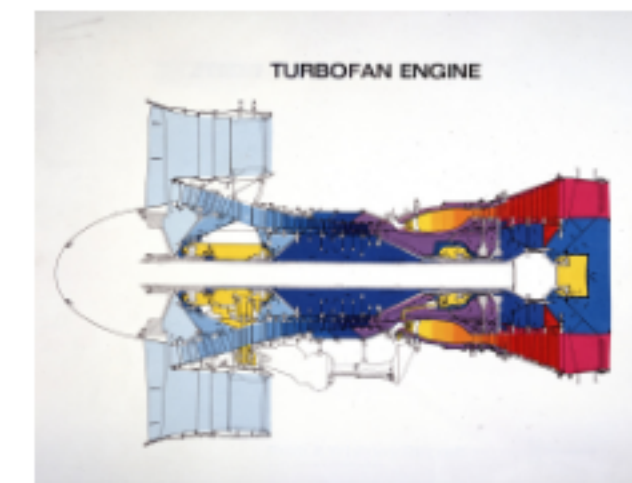
This is NASA C-MAPSS simulated engine degradation data.

Each row is one engine at one operating cycle.

The file contains engine id, cycle number, operating settings, and sensor measurements.

`engine cycle history -> remaining useful life`

The data looks like a table, but the meaning is a set of engine trajectories.



Tools In This Chapter

This chapter stays intentionally local:

- `pandas`: load, inspect, filter, group, and clean tables
- `scikit-learn`: train small models and evaluate them
- `matplotlib`: make plots that reveal data problems
- Markdown reports: keep evidence readable
- small Python scripts: make steps rerunnable

In the labs, the outputs matter as much as the scripts.



The Data Platform We Will Build

Later chapters add shared platform pieces:

- object storage for shared data and artifacts
- Bronze, Silver, Gold layers for data states
- Parquet and table formats for reusable tabular data
- manifest files to version multimedia datasets
- orchestration to run checks and jobs repeatably
- dashboards to inspect prepared analytical data
- MLOps evidence to track models and artifacts

AI-Ready Data Starts Locally

Reliable AI preparation starts before the model.

Remember:

- data shape changes the preparation strategy
- expectations should be explicit
- the split defines what the metric means
- reproducibility needs data, code, transforms, and evidence

Next: move the data path into shared object storage.